

Triggers en SQL Server

Hasta ahora todo lo que hemos visto en el curso requiere que alguien ejecute algo: una consulta, una vista, un ALTER TABLE. Los triggers son diferentes. Un trigger es código que **se ejecuta solo**, automáticamente, cuando algo sucede en una tabla. Nadie tiene que invocarlo. Alguien inserta un registro, actualiza un campo o elimina una fila, y el trigger se dispara por su cuenta.

Son herramientas poderosas. Pero también son peligrosas si se usan mal. Y esa distinción — saber cuándo son la herramienta correcta y cuándo no lo son — es precisamente de lo que trata esta clase.

La analogía que usa el instructor es perfecta: los triggers son como alarmas de incendio. Indispensables en el momento justo, pero un verdadero problema si se disparan cuando no deben. Un trigger bien colocado automatiza el registro de auditoría sin que nadie tenga que hacer nada. Un trigger mal colocado convierte cada INSERT en una cadena oculta de efectos secundarios que nadie recuerda que existen.

Cómo funciona un trigger por dentro

Un trigger se asocia a una tabla y a uno o más eventos: INSERT, UPDATE o DELETE. Cuando ese evento ocurre, SQL Server ejecuta el código del trigger automáticamente. Puede ejecutarlo **después** del evento (AFTER) o **en lugar de** él (INSTEAD OF), dependiendo de cómo lo definas.

Para que el trigger sepa qué datos cambiaron, SQL Server pone a disposición dos tablas especiales que solo existen dentro del contexto del trigger: **INSERTED** y **DELETED**. Su contenido depende del tipo de operación.

Cuando alguien hace un INSERT, la tabla INSERTED contiene las filas nuevas que se acaban de agregar, y DELETED está vacía — porque no se eliminó nada. Cuando alguien hace un DELETE, ocurre lo contrario: DELETED contiene las filas que se eliminaron e INSERTED está vacía. Y cuando alguien hace un UPDATE, las dos tablas tienen contenido: DELETED tiene los valores que había **antes** del cambio e INSERTED tiene los valores **después** del cambio. Pensándolo así, un UPDATE es como si SQL Server eliminara la fila vieja y la reinsertara con los valores nuevos.

Entender estas dos tablas es la clave para escribir cualquier trigger, porque es a través de ellas que accedes a la información de qué cambió.

El caso de uso legítimo: auditoría

De todos los usos posibles de un trigger, hay uno que es ampliamente aceptado como la mejor práctica: **registrar quién cambió qué y cuándo**. Es decir, crear un registro automático de auditoría.

Veamos cómo se construye con TiendaLatam. Lo primero es crear una tabla donde se van a guardar los registros de auditoría — en la clase la llamamos **AuditoriaPedidos**. Esta tabla tiene campos como el ID del pedido que se modificó, qué tipo de acción fue (INSERT, UPDATE o DELETE), el estado y el total antes del cambio, el estado y el total después, quién hizo el cambio y cuándo.

Después viene el trigger propiamente dicho. Se crea con **CREATE TRIGGER**, le das un nombre (la convención es usar el prefijo **TR_**), le indicas sobre qué tabla opera y ante qué eventos se dispara. En nuestro caso: **ON Pedidos AFTER INSERT, UPDATE, DELETE**. Eso significa que cada vez que alguien inserte, actualice o elimine algo en la tabla **Pedidos**, este trigger se ejecuta automáticamente.

Dentro del trigger, la lógica es un INSERT INTO hacia la tabla de auditoría. Usa las tablas INSERTED y DELETED para determinar qué tipo de operación fue — si INSERTED tiene filas pero DELETED no, fue un INSERT; si ambas tienen filas, fue un UPDATE; si solo DELETED tiene filas, fue un DELETE — y registra los valores de antes y después.

En la clase lo probamos en vivo: hicimos un UPDATE cambiando el estado de un pedido a "Cancelado", consultamos la tabla de auditoría, y ahí estaba el registro con el estado anterior, el estado nuevo, el total antes, el total después, el usuario y la fecha exacta. Sin que nadie tuviera que escribir una sola línea de código de registro — el trigger se encargó de todo.

Cuándo NO usar triggers (y esto es igual de importante)

Acá es donde muchos se equivocan. Los triggers son tan convenientes — se ejecutan solos, no requieren cambiar la aplicación — que la tentación es usarlos para todo. Pero tienen limitaciones serias que los hacen problemáticos en muchos contextos.

La primera es que el trigger se ejecuta **una vez por statement, no una vez por fila**. Si haces un UPDATE que afecta 10,000 filas, las tablas INSERTED y DELETED van a contener 10,000 filas cada una. Tu trigger tiene que estar preparado para manejar eso con operaciones sobre conjuntos de datos, no fila por fila. Si no lo diseñas así, el rendimiento se desploma.

La segunda es la invisibilidad. Los triggers no aparecen en el código de la aplicación. Un desarrollador puede estar haciendo un simple INSERT y no tiene idea de que detrás hay un trigger ejecutando lógica adicional. Si algo falla, el error viene de un lugar que nadie estaba mirando. Esa invisibilidad es justamente lo que los hace útiles para auditoría — no quieres que alguien pueda saltarse el registro — pero es lo que los hace peligrosos para cualquier otra cosa.

La tercera es el riesgo de encadenamiento. Un trigger puede disparar otro trigger, que a su vez dispara otro. SQL Server permite hasta treinta y dos niveles de anidación. Si no tienes cuidado, puedes terminar con procesos que se quedan girando en bucle sin terminar.

Por todo esto, hay cosas para las que los triggers definitivamente no son la herramienta correcta. No los uses para **lógica de negocio compleja** — para eso existen los procedimientos almacenados, que son explícitos, testeables y vamos a ver en el próximo módulo. No los uses para **validaciones que se pueden hacer con restricciones** (los CHECK constraints son más eficientes y más claros). No los uses para **replicar datos entre tablas en tiempo real** — hay mecanismos mejores para eso. Y no los uses para **cálculos que pueden resolverse en la consulta** o con columnas calculadas, que ya vimos en la clase anterior.

El criterio es simple: usa triggers para auditoría y para proteger la integridad de los datos. No los uses para lógica que pertenece a la aplicación o al software. Para eso hay otras herramientas, y las vamos a ver en el próximo módulo.

INSTEAD OF: el caso especial

Hay un tipo de trigger que no se ejecuta después del evento sino **en lugar de** él. Se llama INSTEAD OF y su caso de uso más común es interceptar operaciones sobre vistas que normalmente no son actualizables.

Por ejemplo, si alguien intenta hacer un INSERT sobre la vista **VW_VentasPorPais** — que tiene GROUP BY y agregaciones, por lo que no admite escritura — puedes poner un trigger INSTEAD OF INSERT que intercepte esa operación y decida qué hacer: redirigirla a la tabla correcta, rechazarla con un mensaje de error, o lo que tenga sentido para tu caso. Es la única forma de agregar triggers a vistas en SQL Server, y aunque es un caso menos frecuente, es bueno saber que existe.

Desafío de cierre

Diseña e implementa un trigger de auditoría para la tabla **Cientes**. Primero crea una tabla llamada **AuditoriaCientes** donde se van a guardar los registros. El trigger debe registrar qué tipo de operación fue (INSERT, UPDATE o DELETE), el nombre del cliente antes y después del cambio, el estado activo antes y después, quién hizo el cambio y cuándo. Pruébalo con un UPDATE y con un DELETE lógico (cambiar el campo activo de 1 a 0). Comparte tu trigger y los resultados en los comentarios de la clase.